

Probabilistic Machine Learning

Lecture 05: Bayesian Networks

Prof. Dr. Michael Granitzer

Module 1

Belief Networks

Guiding Question: *What exactly does an arrow in a probabilistic model claim — and what does a missing arrow claim?*

Definition

Bayesian belief network (BBN). A **directed acyclic graph** whose nodes are random variables, in which the joint distribution factorises as

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid \text{pa}(x_i))$$

where $\text{pa}(x_i)$ are the parents of x_i in the graph.

Each node carries one conditional probability table given its parents. The wet-grass model was exactly this.

Three Tasks

Modelling

Choose variables, edges and tables that encode the domain.

Inference

Compute posteriors given evidence.
Exact **and** approximate inference are **NP-hard** in general.

Learning

Estimate the tables — and possibly the structure — from data.

Modelling Technique: Divorcing Parents

A node with k binary parents needs 2^k table entries. **Divorcing** inserts an intermediate variable that summarises a group of parents, so the table becomes two small ones instead of one large one.

The intermediate node usually has a meaning of its own — "total load", "combined risk" — which is why the technique improves interpretability as well as size.

Fewer parameters means fewer numbers a domain expert has to invent.

Modelling Technique: Noisy OR

When many causes $x_1, \dots, x_k$ can each independently trigger an effect y , a **noisy OR** models the effect as "each cause fires with its own probability, and the effect occurs if any of them fires":

$$p(y = 0 \mid x_1, \dots, x_k) = \prod_{i:x_i=1} (1 - \lambda_i)$$

k parameters instead of 2^k .

Typical use

Disease–symptom networks, where many diseases give rise to the same symptom.

Uncertain and Unreliable Evidence

Uncertain (soft) evidence

The state of x is itself uncertain, expressed as a distribution $\tilde{x}$. Infer by summing over the states:

$$p(y \mid \tilde{x}) = \sum_x p(y \mid x)p(x \mid \tilde{x})$$

We look at the grass without touching it:

$$p(T=1 \mid \tilde{T}) = 0.8.$$

Unreliable evidence

The evidence is not trusted and is **replaced** by our own estimate:

$$p(y \mid x) \rightarrow p(h \mid x)$$

Jack says his grass is wet, but Jack is in a bad mood and did not check.

Both are drawn with a **dashed** edge.

Takeaway

A belief network is a **DAG plus one conditional table per node**. Modelling techniques — divorcing, noisy OR, soft evidence — all trade assumptions for parameters, and every trade should be stated out loud.

Module 2

Reading Independence off the Graph

*Guiding Question: Two causes are independent until you observe their shared effect.
Which pictures make that visible?*

Three Ways Three Nodes Can Connect

Every conditional-independence question in a DAG reduces to these three patterns.

Chain

$$a \rightarrow c \rightarrow b$$

a and b are dependent — but **independent given c** .

Information flows along the chain until the middle is observed.

Common cause (fork)

$$a \leftarrow c \rightarrow b$$

a and b are dependent — but **independent given c** .

The shared cause explains the correlation.

Common effect (collider)

$$a \rightarrow c \leftarrow b$$

a and b are **independent** — but **dependent given c** .

Explaining away — the reverse of the other two.

Mini-Exercise

For each pair, decide whether observing the middle variable makes the outer two *more* or *less* related:

1. rain $\rightarrow$ wet grass $\rightarrow$ slippery pavement, observing wet grass
2. temperature $\rightarrow$ ice-cream sales, temperature $\rightarrow$ drownings, observing temperature
3. rain $\rightarrow$ wet grass $\leftarrow$ sprinkler, observing wet grass

Mini-Exercise — Solution

Mini-Exercise — Solution

1. Chain

Independent given wet grass
— the pavement only cares
that the ground is wet.

2. Fork

Independent given
temperature — the classic
spurious correlation dissolves.

3. Collider

Dependent given wet grass —
learning it rained lowers belief
in the sprinkler.

d-Separation

A path between X and Y is **blocked** by a set Z if it contains either

- a chain or fork whose middle node **is in** Z , or
- a collider whose middle node — and all of its descendants — are **not in** Z .

If every path from X to Y is blocked by Z , then X and Y are **d-separated** given Z , and $X \perp \perp Y \mid Z$.

d-separation is sound: what it declares independent really is independent for every distribution factorising over the graph. It is not complete for a *particular* distribution — extra independences may hold numerically without being visible in the graph.

Takeaway

Independence in a belief network is a **graph property**: chains and forks are blocked *by* observing the middle node, colliders are opened by it. d-separation is that rule applied to every path.

Module 3

Graphical Notation

Guiding Question: *A model of 10,000 observations should not need 10,000 nodes. How is a repeated structure drawn?*

Drawing a Model

A probabilistic model is a factorisation, and the graph is its picture. Four conventions carry almost all the information:

Shaded node

An **observed** variable — its value is in the data.

Unshaded node

A **latent** variable or a parameter — inferred, never seen.

Small solid dot

A fixed **hyperparameter**, not inferred.

Arrow

A conditional dependence: the child's distribution is conditioned on the parent.

Plates

A **plate** is a rectangle drawn around part of the graph with a count in its corner: *repeat this structure N times, with independent copies of the variables inside.*

It is the i.i.d. assumption, drawn. Without it, a model of N observations would need N copies of the same subgraph.

Nested plates express grouped data — N documents each of M_n words, or K hospitals each with N_k patients.

What sits outside the plate

Anything **shared** across observations — the parameters. That is precisely what learning estimates.

Reading a Diagram

For a Bayesian linear regression: a hyperparameter dot feeds $\mathbf{w}$, which sits **outside** the plate inside a plate over N observations sit $\mathbf{x}_i$ (shaded) and y_i (shaded), with $\mathbf{w}$ pointing into each y_i .

Read off the factorisation directly: $p(\mathbf{w} \mid \tau) \prod_{i=1}^N p(y_i \mid \mathbf{x}_i, \mathbf{w})$. The plate became the product the arrows became the conditioning bars.

Mini-Exercise

A diagram shows a dot $\rightarrow \alpha \rightarrow \theta$ inside a plate of K , and $\theta \rightarrow z_i \rightarrow x_i$ inside a plate of N , with x_i shaded.

1. Which variables are inferred?
2. What does the N plate assert?
3. Write the factorisation.

Mini-Exercise — Solution

Plate notation compresses a factorisation into a diagram: **shading** marks what was observed, **plates** mark what repeats independently, and **what sits outside the plates** is what learning has to estimate.

Mini-Exercise — Solution

1. θ and every z_i — unshaded. Only x_i is observed α is fixed.

2. The N observations are **conditionally independent given θ** — each with its own latent z_i .

3. $p(\theta \mid \alpha) \prod_{i=1}^N p(z_i \mid \theta) p(x_i \mid z_i)$ — a mixture model.

Plate notation compresses a factorisation into a diagram: **shading** marks what was observed, **plates** mark what repeats independently, and **what sits outside the plates** is what learning has to estimate.

Module 4

Undirected Models

Guiding Question: *Pixels influence each other symmetrically — no pixel causes its neighbour. How do we model that?*

Undirected Models

Markov network (Markov random field). An **undirected** graph in which the joint distribution factorises over the **cliques** $c \in \mathcal{C}$ of the graph:

$$p(x_1, \dots, x_n) = \frac{1}{Z} \prod_{c \in \mathcal{C}} \phi_c(x_c), \quad Z = \sum_x \prod_c \phi_c(x_c)$$

The $\phi_c \geq 0$ are **potentials** — arbitrary non-negative functions, not probabilities. Z is the normalising **partition function**.

When all cliques have size 2 the model is a **pairwise Markov network**.

Directed vs. Undirected

Belief network

Directed, acyclic. Factors are conditional probabilities. Natural for **causal or generative** stories conditioning on a collider creates dependence.

Markov network

Undirected. Factors are potentials with a global Z . Natural for **symmetric interactions** — pixels, atoms, neighbours in a lattice.

Marginalising and Conditioning Behave Oppositely

Marginalising over C

makes A and B **dependent** — the sum over all states of c couples the potentials.

Conditioning on C

In the chain $A - C - B$, a fixed c is a constant in the potentials, so the factor splits and $A \perp\!\!\!\perp B \mid C$.

This holds because C separates them — not for every C .

Markov Properties and the Separation Test

For undirected models, independence is plain graph separation: if every path from X to Y passes through Z , then $X \perp\!\!\!\perp Y \mid Z$.

For a **directed** model the same question is answered by converting the graph:

1. Ancestral graph

Keep $X \cup Y \cup Z$ and their ancestors delete all other nodes and their edges.

2. Moralisation

Connect any two nodes that share a child, then drop all arrow heads.

3. Separation

Remove the links neighbouring Z . If no path joins X to Y , then $X \perp\!\!\!\perp Y \mid Z$.

Two Examples

Boltzmann machine

A pairwise Markov network over binary units with weights on the edges the restricted variant (RBM) allows edges only between a visible and a hidden layer.

Image cleanup

Pixels form a lattice: each potential prefers a pixel to agree with its neighbours and with the noisy observation. Inference denoises the image.

Other Graphical Models

Chain graphs

Combine directed and undirected edges in one model.

Factor graphs

Make each factor an explicit node — mostly used **for inference**, since they state precisely which variables each factor touches.

Undirected models trade causal direction for symmetric interaction, and local normalisation for a global partition function. Separation in the (moralised) graph is the single test for conditional independence.

Module 5

Fitting and Inference

Guiding Question: *We can now write the model down. Can we actually compute anything with it?*

Three Questions You Can Ask a Model

Infer unobserved variables

Compute the posterior given evidence — probabilistic inference.

Learn the parameters

Estimate the factors' distributions from data.

Learn the structure

Discover which dependencies exist at all.

The three get progressively harder — and progressively more interesting.

Inference Is Hard in General

Both exact and approximate inference in belief networks are **NP-hard**. Whether a particular model is tractable depends on its structure.

Exact inference

Variable elimination, junction trees, message passing.
Efficient when the graph is a tree or has low treewidth.

Approximate inference

Sampling (MCMC, importance sampling) and variational methods.
Trade a guarantee for tractability.

Learning the Parameters

With **fully observed** data, parameter learning is counting: each conditional table is estimated from the frequencies of its variable and parents.

With **hidden variables**, the counts are unavailable and one alternates between inferring the hidden values and re-estimating the parameters (expectation–maximisation).

The estimates are only as good as the sample supporting each table row. A parent combination that never occurs in the data leaves its row undetermined — which is another argument for keeping tables small.

Takeaway

Graphical models make the joint distribution **writable** whether they make it **queryable** depends on the graph's structure. Parameters come from counting where data is complete, and from iterating where it is not.